

Plantilla de Especificación de Requisitos - Proyecto Final IoT

Equipo: Grokisthistrue?

Fecha de última actualización: 04/06/2026

1. Introducción

Sistema de Monitoreo para Invernadero Inteligente

Se desea desarrollar un sistema IoT para el monitoreo en tiempo real de un invernadero. El sistema recopila telemetría clave del entorno utilizando tres modalidades de entrada distintas para garantizar condiciones óptimas de cultivo:

- **Humedad del suelo:** Monitoreo analógico de la tierra para evaluar de forma precisa la necesidad hídrica de las plantas.
- **Clima ambiental:** Medición digital de la temperatura y la humedad del aire dentro del habitáculo para mantener el entorno de crecimiento ideal.
- **Nivel de reserva hídrica:** Control por sensor de distancia del estado del tanque de agua, diseñado estratégicamente para prevenir que la bomba de riego se queme por funcionar en vacío.

Toda esta información es adquirida por un microcontrolador (ESP32), procesada por un backend, persistida en una base de datos histórica y expuesta al usuario mediante un panel de control interactivo.

Casos de uso:

Monitoreo en tiempo real

Historia de Usuario:

- **Como** usuario de mi invernadero
- **Quiero** ver en un dashboard los datos actuales de humedad del suelo, la temperatura del ambiente y el nivel del agua
- **Para** conocer el estado que se encuentran mis plantas

Criterio de aceptación:

- **Dado** que estoy en la pantalla principal del dashboard
- **Cuando** el sistema recibe una nueva medición
- **Entonces** el valor se actualiza automáticamente sin que yo tenga que recargar

Consulta de historial

Historia de Usuario:

- **Como** usuario de mi invernadero
- **Quiero** consultar en el dashboard el historial de mediciones
- **Para** poder trazar las condiciones a las cuales fueron sometidas las plantas

Criterio de aceptación:

- **Dado** que el sistema guarde las mediciones en una base de datos
- **Cuando** me voy a la sección historial
- **Entonces** me muestra una lista de las mediciones con el formato fecha, hora y los valores exactos de cada sensor

Transmisión de datos

Historia de Usuario:

- **Como** usuario del sistema
- **Quiero** que el ESP 32 envíe un paquete con los datos obtenidos de los sensores
- **Para** subir la información a una base de datos

Criterio de aceptación:

- **Dado** que obtuve las mediciones de los sensores
- **Cuando** envío la información
- **Entonces** el sistema deberá guardarla en una base de datos

Datos corruptos

Historia de Usuario:

- **Como** usuario del sistema
- **Quiero** que mi servidor backend valide los datos ingresados
- **Para** Tener persistencia en mi base de datos

Criterio de aceptación:

- **Dado** que el backend expone un endpoint
- **Cuando** recibe un dato en un mal formato
- **Entonces** debe devolver un http 400 Y no añadirlo a la base de datos

Falla conexión con la ESP32

Historia de Usuario:

- **Como** usuario del sistema
- **Quiero** recibir un aviso de que falló la conexión con la ESP32
- **Para** evitar que el usuario visualice vieja información

Criterio de aceptación:

- **Dado** que el backend recibió un dato
- **Cuando** no recibe datos durante largos periodos de la ESP32
- **Entonces** mostrar una notificación de falla de conexión

2. Requerimientos Funcionales (RF)

- **RF1 - Adquisición de datos mediante sensores:** El firmware de la ESP32 debe leer de manera periódica las señales de los sensores físicos correspondientes a las tres modalidades especificadas: humedad del suelo (analógica), clima ambiental (digital) y nivel de reserva hídrica (distancia).

- **RF2 - Transmisión de Telemetría:** La ESP32 debe empaquetar las lecturas obtenidas y enviarlas de forma inalámbrica hacia el servidor backend mediante peticiones HTTP REST.
- **RF3 - Validación Estricta de Payload (Backend):** El servidor backend debe validar el formato y la integridad estructural de cada paquete de datos entrante antes de procesarlo.
- **RF4 - Persistencia de Histórico:** El backend debe almacenar de forma permanente todas las mediciones que hayan sido validadas correctamente.
- **RF5 - Panel de Control en Tiempo Real (Dashboard):** El frontend debe proveer una interfaz gráfica interactiva que muestra los valores actuales de humedad de suelo, temperatura ambiental y nivel del tanque de agua.
- **RF6 - Actualización Reactiva de la UI:** La interfaz de usuario debe actualizar automáticamente los componentes visuales del dashboard de manera inmediata al recibir nuevas mediciones desde el backend, omitiendo la necesidad de una recarga manual de la página (*refresh*).
- **RF7 - Consulta de Datos Históricos:** El frontend debe ofrecer una sección dedicada a listar las mediciones almacenadas en el sistema, mostrando explícitamente para cada registro los campos de: fecha, hora y el valor exacto del sensor.
- **RF8 - Control de Excepciones por Datos Corruptos:** Ante la recepción de datos malformados o con formato erróneo, el backend debe rechazar la solicitud de inmediato emitiendo un código de estado HTTP 400 (Bad Request) y garantizar que dicha información basura no sea añadida a la base de datos.
- **RF9 - Alerta por Pérdida de Conexión:** El backend debe monitorear el tiempo transcurrido desde el último reporte de la ESP32. Si el dispositivo embebido deja de transmitir datos por un periodo largo de tiempo, el sistema debe disparar una notificación visual de falla de conexión en el dashboard del usuario, manteniendo visibles los últimos datos conocidos para evitar mostrar pantallas vacías.

3. Requerimientos No Funcionales (RNF)

- **RNF1 - Arquitectura de Red Unidireccional Estricta:** El flujo general de integración de componentes debe respetar de forma obligatoria la cadena: ESP32 → Backend → PostgreSQL, Backend → Frontend. El frontend debe consumir los datos de dominio pura y exclusivamente a través de la API REST del backend. Queda estrictamente prohibida cualquier comunicación directa entre el Frontend y la ESP32.
- **RNF2 - Restricción del Motor de Base de Datos:** El sistema de persistencia histórica del invernadero debe implementarse mandatoriamente utilizando el motor relacional PostgreSQL.
- **RNF3 - Idioma de Desarrollo Técnico:** Todo el código fuente, la estructura de la base de datos, los comentarios técnicos dentro de los archivos y la definición de las propiedades internas deben escribirse exclusivamente en idioma inglés.
- **RNF4 - Acoplamiento por Contrato de API:** La comunicación entre el hardware, el backend y el frontend debe regirse de manera estricta y sin excepciones bajo el esquema de datos documentado en el contrato OpenAPI formalizado en el repositorio (*openapi.json*).

- **RNF5 - Integración Continua Automatizada (CI):** El repositorio del proyecto debe contar con flujos de trabajo configurados en GitHub Actions que compilen y ejecuten de forma automática los tests unitarios del firmware, backend y frontend ante la apertura de cada Pull Request hacia la rama de integración.
- **RNF6 - Aseguramiento de Calidad de Código:** Se debe configurar y ejecutar de forma obligatoria una herramienta de análisis estático (*linter*) integrada en los hooks de pre-commit de Git para validar las reglas de estilo antes de permitir subir cambios al repositorio.
- **RNF7 - Stack Tecnológico y Lenguajes:** El ecosistema debe desarrollarse bajo las siguientes tecnologías: el firmware de la placa sensora debe ser programado en **C/C++** (usando PlatformIO), el servidor backend debe estar construido en **TypeScript** utilizando el framework NestJS (Node.js 18+), y el panel de control (frontend) en un framework moderno de JavaScript.
- **RNF8 - Procesamiento de Algoritmos:** El servidor backend debe ejecutar de manera obligatoria al menos tres algoritmos distintos sobre la información recolectada
- **RNF9 - Rendimiento y Latencia:** El flujo de telemetría debe ser lo suficientemente ligero para que el tiempo transcurrido desde que la ESP32 emite el paquete HTTP hasta que el panel de control web se actualiza no supere los 3 segundos en condiciones de red normales.